

Pipeline CI/CD de inventario-app

Sistemas Distribuidos

Alexander Chuquipoma, Jean Pierre Valarezo

24 de julio de 2026

AlexChuquipoma/inventario-app

Resultado. Pipeline fail-fast con pruebas, Docker multi-stage, publicación en GHCR, RollingUpdate, Blue-Green y los **tres componentes adicionales:** Secret, Trivy y readiness con arranque lento.

Estrategia elegida: Blue-Green

Elegimos Blue-Green porque /version expone versión, color y hostname. Esto permite validar Green antes del corte y comprobar de forma determinista qué versión recibe el tráfico. El Service cambia su selector entre slot: blue y slot: green; el cambio y el rollback son inmediatos y usan solo Deployment y Service nativos de Kubernetes.

En esta aplicación, Canary habría hecho la demostración más probabilística y más difícil de interpretar porque cada pod mantiene su propio archivo JSON. El costo de ejecutar temporalmente dos ambientes es aceptable en Minikube.

Métricas DORA propias

00:11:48 Lead time promedio (5 cambios)	7 / día Frecuencia (7 éxitos, 1 día)	12,5 % Change failure rate (1 de 8 intentos)
--	---	---

Commit	Cambio	Commit UTC	Clúster UTC	Lead time
0568410	Pipeline e imagen inicial	17:08:05	17:36:52	00:28:47
117567a	Rolling deployment	17:58:11	18:02:16.297	00:04:05.297
1077675	Arranque lento y probes	18:40:20	18:50:19.821	00:09:59.821
99d4de3	Secret API_KEY	18:58:22	19:03:31	00:05:09
30fb843	Trivy y runtime seguro	19:24:37	19:35:36	00:10:59

Metodología. Se normalizaron a UTC los timestamps de Git y los momentos observados o registrados por Kubernetes. La frecuencia cuenta siete promociones exitosas que crearon o actualizaron Deployments. Los cortes del Service Blue-Green no crean revisiones y se registraron aparte. El único intento fallido fue el primer Deployment base; requirió una corrección. El rollback Blue-Green fue planificado y no se clasificó como fallo.

Persistencia y comportamiento distribuido

El producto **POD-001**, creado directamente en un pod, desapareció al eliminar ese pod y esperar su reemplazo. **data/products.json** vive en la capa efímera de cada contenedor: no es almacenamiento compartido y dos réplicas pueden mostrar catálogos diferentes. En producción se necesitaría una base externa o una estrategia de almacenamiento compatible con múltiples réplicas; aumentar réplicas por sí solo no soluciona la persistencia.

Problemas reales y soluciones

1. Identidad non-root El primer rollout quedó en CreateContainerConfigError . Aunque la imagen usaba USER node , Kubernetes no podía comprobar un usuario no numérico. Se mantuvo runAsNonRoot: true y se declararon runAsUser/runAsGroup: 1000 .
2. Vulnerabilidad crítica Trivy encontró CVE-2026-59873 en tar 7.5.15 , arrastrado por el npm global del runtime. npm/npx se conservaron en build para instalar, probar y podar dependencias, y se eliminaron del runtime. El reescaneo terminó con código 0 y la app siguió respondiendo 200.
3. Selectores superpuestos kubectl exec deployment/inventario-app podía seleccionar un pod Blue-Green por compartir app=inventario-app . Las verificaciones del Deployment base pasaron a usar app=inventario-app,!slot , haciendo inequívoca la evidencia.

Tres buenas prácticas implementadas

Secret API_KEY por secretKeyRef; Git no contiene el valor.	Trivy CRITICAL bloquea los push de commit y latest.	Readiness realista 503 inicial, startupProbe y tráfico solo al estar listo.
--	---	---

Reflexión final

La automatización útil no termina cuando una imagen compila. Las pruebas reducen errores funcionales; Trivy evita publicar una imagen crítica; los probes impiden enviar tráfico a procesos aún no listos; el Secret separa la credencial del código; y Blue-Green limita el riesgo del cambio de versión. DORA hizo visible el tiempo completo hasta el clúster, no solo la duración del workflow. El aprendizaje principal fue poder reproducir tanto los éxitos como los fallos y explicar por qué ocurrieron.